8-dars

Model Building

Class in Data Preprocessing

Data Preprocessing ni Class orqali bajarish

```
# missing
def tozala(df):
    for col in df.columns:
        if df[col].isnull().any():
            if df[col].dtype == 'object':
                df[col] = df[col].fillna(df[col].mode()[0], inplace=True)
            else:
                df[col] = df[col].fillna(df[col].mean(), inplace=True)
    return df
tozala(df)

# encoding
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
def encodla(df):
    for col in df.columns:
        if df[col].dtype == 'object':
            if df[col].nunique() <= 5:
                dummies = pd.get_dummies(df[col], prefix=col, dtype=int)
                df = pd.concat([df.drop(columns=[col]), dummies], axis=1)
            else:
                df[col] = encoder.fit_transform(df[col])
    return df

# scaler
from sklearn.preprocessing import MinMaxScaler
def scale_qil(df):
    scaler = MinMaxScaler()
    for col in df.columns:
        num_col = df.select_dtypes(include=['int64', 'float64']).columns.drop('x6')
        df[num_col] = scaler.fit_transform(df[num_col])
    return df
```

Class + Data Preprocessing

```
class DataPreProcessing:
    def __init__(self, df):
        self.df = df

#missing ⇒  def tozala(df):  ⇒  def tozala(self):
        for col in self.df.columns:
            if self.df[col].isnull().any():
                if self.df[col].dtype == 'object':
                    self.df[col].fillna(self.df[col].mode()[0], inplace=True)
                else:
                    self.df[col].fillna(self.df[col].mean(), inplace=True)
        return df  ⇒  return self

#encoding ⇒  def encodla(df):  ⇒  def encodla(self):
        encoder = LabelEncoder()
        for col in self.df.columns:
            if self.df[col].dtype == 'object':
                if self.df[col].nunique() <= 5:
                    dummies = pd.get_dummies(self.df[col], prefix=col, dtype=int)
                    self.df = pd.concat([self.df.drop(columns=[col]), dummies], axis=1)
                else:
                    self.df[col] = encoder.fit_transform(self.df[col])
        return df  ⇒  return self

#scaling ⇒  def scale_qil(df):  ⇒  def scale_qil(self):
        scaler = MinMaxScaler()
        for col in self.df.columns:
            num_col = self.df.select_dtypes(include=['int64', 'float64']).columns.drop('x6')
        self.df[num_col] = scaler.fit_transform(self.df[num_col])
        return df  ⇒  return self
```

!!! yuqoridagi kodda biz $\boxed{\text{df}}$ ni ⇒ $\boxed{\text{self.df}}$ ga o'zgartirishimiz k/k

$\boxed{\text{ctrl + f2}}$

```
dp = DataPreprocessing(df)
dp.tozala().encodla().scale_qil()
preprocessed_data = dp.df
```

Buni .py papkada yozildi